Team Roles and Responsibilities

Sports Club Management System — CS323 Database Systems

David Acheampong Awuah, Richard Yemoh, Samira Donkoh, Ronald Ocloo

---

1. How we divided the work

The project runs across ten phases and we are four people, so we assigned ownership by phase rather than by table. Each phase has one name against it. When something in that phase is wrong or late, there is no question about who fixes it.

Two phases are shared between two of us. Rather than both working on the same files, we split them by subject: Samira takes everything that runs inside the database as code, and Ronald takes everything to do with access and recovery. Phase 8, the application, is the one part we all build, and we have divided it by screen area so that no two of us are editing the same files.

Member	Role	Phases owned	Main deliverable
David Acheampong Awuah	Database and Schema Lead	3 and 4	Normalized schema and the DDL scripts everything else is built on
Richard Yemoh	Data and Query Lead	5 and 6a	Seed data, advanced queries and views
Samira Donkoh	Database Programming Lead	6b	Stored procedures, functions and triggers
Ronald Ocloo	Database Security Lead	7	Roles, privileges, backup and recovery
All four members	Application	8	CRUD application with search, reporting and role-based access

2. Individual responsibilities

2.1 David Acheampong Awuah, Database and Schema Lead (Phases 3 and 4)

David converted the Phase 2 entity-relationship model into the normalized 12-table schema and writes the DDL that creates it.

Deliverables

- Phase 3 logical design: normalization to Third Normal Form, the data dictionary and the updated ER diagram
- The three schema scripts: 01_create_database.sql, 02_tables.sql and 03_indexes.sql
- All constraints: primary keys, foreign keys with explicit delete rules, CHECK constraints, ENUM domains, and the UNIQUE constraint that prevents facility double-booking
- The eight indexes, each justified against a stated requirement
- Technical documentation covering schema and DDL decisions

Why one person owns the schema

From Day 2 onward, three of us are writing SQL against the same tables at the same time. If two people change a column definition in different branches, the merge is silent and the breakage shows up somewhere else entirely. Routing every schema change through one person means there is always a single correct version.

What the rest of us depend on

The DDL merging into the main branch at the end of Day 1 is the bottleneck for the whole project. Until the tables exist, Richard cannot insert data, Samira cannot compile a procedure, and none of us can point application code at anything. David tags that merge so we have a fixed reference to compare against if the schema drifts later.

Schema change protocol

If any of us hits a gap, such as a missing column or a constraint that blocks a legitimate case, we do not change it ourselves. We tell David, he decides, and he announces the change in the group chat before pushing it, because it affects everyone's work in progress.

Part in the application

Database connection layer and configuration. He also acts as integrator on Day 4, since his Phase 4 workload finishes earliest.

2.2 Richard Yemoh, Data and Query Lead (Phases 5 and 6a)

Richard populates the database and writes the queries and views that everything else reports from.

Deliverables

- Seed data of 20 to 30 records per entity, inserted in foreign key order: parent tables first, then dependent tables, then the weak entities
- At least ten advanced queries using joins, subqueries, aggregates and grouping
- Five views covering common reporting needs, such as active members, revenue by period and upcoming bookings
- Test cases for his own queries in Phase 9

Why the data matters more than it looks

Queries cannot be tested properly against three rows, and neither can indexes. The dataset needs realistic spread: memberships in all three statuses, payments that are complete and pending, bookings on the same facility on different days, and athletes on more than one team. If everything in the seed data is clean and current, half our queries return nothing at the demonstration and we cannot show that the constraints work.

Working around the Day 1 dependency

Richard cannot insert anything until the DDL exists, so on Day 1 he designs the dataset on paper: which athletes join which teams, which memberships have lapsed, which payments are outstanding. When the schema lands he is typing, not deciding.

Part in the application

Search and filter screens (FR8) and the reporting screens (FR9), since both are built directly on his queries and views.

2.3 Samira Donkoh, Database Programming Lead (Phase 6b)

Samira writes the logic that runs inside the database rather than in the application.

Deliverables

- **Triggers.** Business Rule 1, which requires an athlete to hold a membership before being added to a team roster. This cannot be done with a foreign key, because it is a conditional rule across two tables. Also a trigger preventing an athlete from holding two Active memberships at once, which a UNIQUE index cannot express.
- **Stored procedures.** Registering an athlete with a membership and first payment as one transaction; renewing a membership as a new row rather than an edit; recording a payment and updating its status.
- **Functions.** Outstanding balance on a membership, athlete age from date of birth, and active roster count for a team.
- Test cases proving each trigger fires and each procedure rolls back correctly on failure

Two things to watch

A trigger that only fires on INSERT is bypassed by an UPDATE, so the same rule needs to be enforced on both. Procedures that touch more than one table should either commit fully or roll back fully; registering an athlete and then failing to record their membership leaves the database in a state that Business Rule 1 says should be impossible.

Part in the application

CRUD screens for the core entities, calling her own procedures rather than writing raw INSERT statements in application code.

2.4 Ronald Ocloo, Database Security Lead (Phase 7)

Ronald controls who can reach what, and makes sure we can rebuild the database if something goes wrong.

Deliverables

- CREATE ROLE and GRANT statements for three roles: admin, coach and front-desk
- Privileges assigned on least privilege. Front-desk staff can record a payment but not alter a completed one, and a coach can read their own team's roster but not touch financial tables. This is what satisfies Business Rule 7.

- A backup and recovery strategy using mysqldump, with a documented restore that we have actually run rather than only described (NFR3)
- The password hashing approach for APP_USER.password_hash (NFR2)

A distinction worth making in the presentation

There are two separate layers of access control in this project and they are easy to confuse. Ronald's GRANT statements secure the database, deciding what a connected database user is allowed to touch. The role column in APP_USER secures the application, deciding what a logged-in member of staff sees on screen. Both are required, and explaining why is the kind of question we should expect to be asked.

Part in the application

Login, role-based access control and input validation.

3. The application, Phase 8

This is the only phase all four of us work on. To keep us out of each other's files, we split it by area.

Area	Owner	Depends on
Database connection layer and configuration	David	The schema and the environment template
CRUD screens for all entities	Samira	Her stored procedures
Search, filtering and reporting screens	Richard	His queries and views
Login, role-based access and input validation	Ronald	APP_USER and his role definitions
Integration and merge on Day 4	David	All of the above

We named an integrator because work that belongs to everyone tends to belong to nobody. Phase 8 is the piece most likely to run over, so one of us has to be responsible for the parts actually fitting together. David takes it because his own phases finish first.

4. Testing, documentation and presentation, Phases 9 and 10

Each of us writes test cases for our own work and documents our own section. One person compiles the pieces into the final submission.

Item	Owner	Covers
Schema and DDL documentation, data dictionary	David	Phases 3 and 4
Query test cases and sample outputs	Richard	Phases 5 and 6a
Procedure and trigger test cases	Samira	Phase 6b
Security testing and restore test evidence	Ronald	Phase 7
User manual and final compilation	Richard	Phases 9 and 10
Limitations and future enhancements	All four	One section each

For the demonstration, each of us presents the phases we own. The design justification at the start draws on the Phase 1 to 3 notes, so we can trace every table back to a numbered requirement rather than explaining it from memory.

5. How we work together

- Nobody pushes to the main branch directly. We branch per phase and open a pull request.
- The main branch is protected and needs a review before merging, so a broken script cannot reach the branch everyone is working from.
- We pull the main branch at the start of every session, before writing anything.
- Schema changes go through David and are announced before they are pushed.
- Credentials never go in the repository. The environment file is ignored by git, and a template file shows which variables to set.

- The database is rebuildable from scripts alone. If anyone's local copy breaks, they drop it and re-run the schema scripts and then the data script.